Relatório Técnico — MGPEB

Módulo de Gerenciamento de Pouso e Estabilização de Base · Nave Aurora Siger · Fase 2 · Grupo 42

Integrantes: Ronei Davi Lach, Gabriel Hampel Meireles, Roberto Kenji Iuvata, José Roberto Zendron e David Silva

Repositório: https://github.com/rdlach/fiap-fase2-projeto-aurora-siger

1. Introdução e cenário de pouso

Na Fase 1, a Aurora Siger precisava decidir se estava pronta para decolar. Nesta fase, a nave já está na órbita de Marte e libera módulos automatizados que descem sozinhos até a superfície para formar a base inicial da colônia. Como o atraso de comunicação com a Terra impede o controle humano em tempo real, desenvolvemos o MGPEB (Módulo de Gerenciamento de Pouso e Estabilização de Base): um protótipo em Python que organiza os módulos em fila, aplica regras lógicas de autorização e classifica cada módulo como POUSADO, ESPERA ou ALERTA.

Definimos cinco módulos, cada um com os atributos pedidos no enunciado: prioridade de pouso (1 é a mais urgente), nível de combustível, massa, criticidade da carga (5 é a mais crítica) e horário estimado de chegada à órbita. A tabela mostra os valores do modo base, o cenário fixo usado na demonstração.

Módulo	Carga	Prioridade	Combustível	Massa	Criticidade	Órbita	Sensores
M01 Habitat Orion	Habitação	1	62%	9.200 kg	5	08:10	Funcionais
M02 Gerador Helios	Energia	1	54%	8.700 kg	5	08:18	Funcionais
M03 Lab Ares	Ciência	3	33%	7.600 kg	3	08:24	Funcionais
M04 Carga Atlas	Logística	4	28%	10.800 kg	2	08:32	Funcionais
M05 Suporte Vita	Médica	2	47%	6.900 kg	4	08:40	Falha parcial

Além dos módulos, o cenário simula as condições de Marte no momento do pouso. No modo base, a atmosfera está estável, o vento é de 12 m/s, a visibilidade é de 8 km, não há tempestade de poeira e a área de pouso está livre. O programa oferece três modos de dados: base (valores fixos, para demonstração), randômico (valores sorteados, para testar as regras em situações variadas) e manual (valores digitados, para forçar um cenário específico).

2. Regras de decisão e portas lógicas

Cada módulo é avaliado por cinco verificações booleanas, calculadas na função `calcularChecks`, e por uma condição crítica usada apenas na regra de alerta.

Variável	Nome no código	Verdadeira quando
A	combCheck	combustível ≥ 35%
B	`atmCheck`	atmosfera estável, vento ≤ 18 m/s, visibilidade ≥ 4 km e sem tempestade de poeira
C	`areaCheck`	área de pouso livre
D	sensCheck	sensores funcionais
E	massaCheck	massa ≤ 12.000 kg
K	(falha crítica)	combustível < 30%

A partir dessas variáveis, modelamos três funções booleanas:

```
AUTORIZAR = A · B · C · D · E
ALERTA    = (A' · K) + D'   =   K + D'
ESPERA    = (AUTORIZAR + ALERTA)'
```

A autorização usa uma porta AND de cinco entradas: basta uma condição falsa para o pouso não ser liberado. A verificação atmosférica B é, ela própria, uma porta AND de quatro condições ambientais. A regra de alerta foi simplificada: como K (combustível abaixo de 30%) já implica A' (combustível abaixo de 35%), o termo A' não altera o resultado, e ALERTA reduz-se a K + D', ou seja, combustível crítico OU sensores com falha. A espera é a saída de uma porta NOR: o módulo aguarda quando não foi autorizado nem está em alerta.

As três saídas são mutuamente exclusivas. Se AUTORIZAR = 1, então A = 1 e D = 1; logo K = 0 e D' = 0, o que torna ALERTA = 0. Pela definição de ESPERA, ela só vale 1 quando as outras duas valem 0. Assim, cada módulo recebe exatamente uma classificação. A Figura 1 mostra o circuito completo e os casos do modo base.

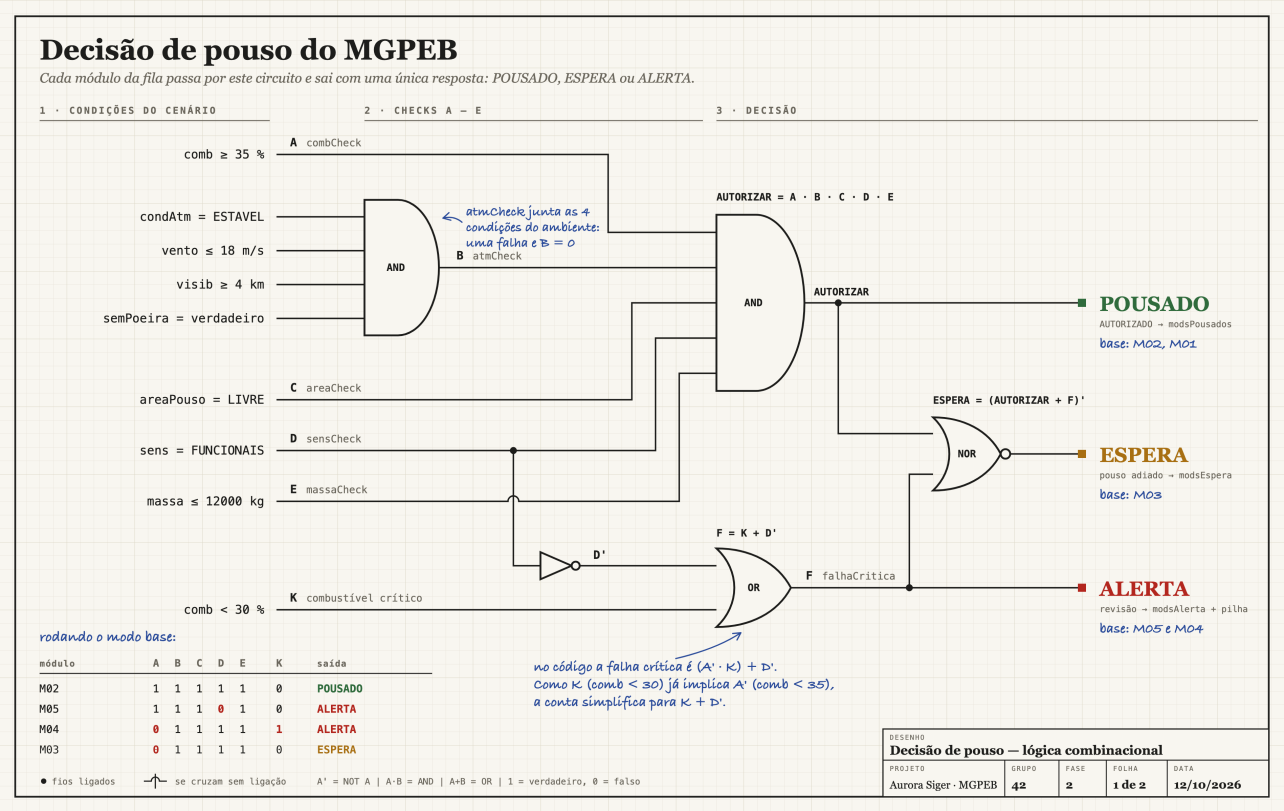


Figura 1 – Circuito lógico da decisão de pouso (portas AND, OR, NOT e NOR) com os casos do modo base

Caso	A	B	C	D	E	K	Saída
M02 e M01 (modo base)	1	1	1	1	1	0	POUSADO
M05 (modo base)	1	1	1	0	1	0	ALERTA
M04 (modo base)	0	1	1	1	1	1	ALERTA
M03 (modo base)	0	1	1	1	1	0	ESPERA

O modo base cobre as três saídas. M03 e M04 mostram a diferença entre ESPERA e ALERTA: os dois falham em A, mas M03, com 33% de combustível, está abaixo do mínimo de 35% e acima do nível crítico de 30% (K = 0), por isso aguarda; M04, com 28%, está abaixo do nível crítico (K = 1) e vai para revisão.

3. Protótipo em Python

O protótipo está no arquivo `aurora_siger_pt2.py`, na pasta `projeto2-grupo`, e é executado pelo terminal com o comando `python3 projeto2-grupo/aurora_siger_pt2.py`. Ele usa apenas recursos básicos da linguagem e o módulo `random`, para o modo randômico, sem bibliotecas avançadas. Um menu controlado por um laço while permanece ativo até a opção 0 e oferece: regras lógicas (1), buscas (2), carga e ordenação da fila (3), processamento da fila (4), resumo final (5), modelo matemático (6), demonstração completa (7) e escolha do modo de dados (8).

3.1 Cadastro e estruturas lineares

Cada módulo é um dicionário com seus atributos. Ao carregar a simulação, os cinco módulos são copiados para a fila principal filaPouso, que funciona como fila FIFO. Durante o processamento, cada módulo é direcionado a uma de três listas auxiliares (modsPousados, modsEspera ou modsAlerta), e os alertas também são registrados em uma pilha LIFO (pilhaAlertas). O Anexo A detalha o uso de cada estrutura.

3.2 Ordenação da fila

A fila é reorganizada pela função `ordenarFilaPorPrio`, que implementa o algoritmo de ordenação por inserção (insertion sort). O critério, definido na função `vemDepois`, compara em três níveis: menor número de prioridade primeiro; em empate, maior criticidade primeiro; em novo empate, menor combustível primeiro. O último critério atende diretamente à ideia de priorizar módulos com pouca reserva de combustível. No modo base, M01 e M02 empatam em prioridade e criticidade, e M02 desce antes por ter menos combustível (54% contra 62%). A fila ordenada fica M02, M01, M05, M03, M04.

Escolhemos o insertion sort por ser simples, ordenar deslocando elementos dentro do próprio vetor e ter bom desempenho em listas pequenas ou quase ordenadas. No protótipo, a função trabalha sobre uma cópia da fila, para preservar o cadastro original. Sua complexidade de pior caso é $O(n^2)$, o que não é um problema para cinco módulos; em uma frota muito maior, um algoritmo $O(n \log n)$, como merge sort ou heap sort, seria mais adequado.

3.3 Buscas

As três buscas usam busca linear, que percorre a lista do início ao fim com complexidade O(n) e não exige que os dados estejam ordenados. No modo base, os resultados são:

Função	Busca	Resultado
buscarMenorComb	módulo com menor combustível	M04 Carga Atlas (28%)
buscarMaiorPrio	módulo mais urgente (desempate pela criticidade e, depois, pelo menor combustível)	M02 Gerador Helios
buscarPorTipo	todos os módulos de um tipo (ENE)	M02 Gerador Helios

A busca de maior prioridade usa a mesma comparação da fila (`vemDepois`); por isso, as duas apontam M02. A busca binária, de complexidade O(log n), não foi usada porque exigiria manter a lista ordenada pelo atributo pesquisado, e o ganho seria irrelevante para cinco elementos.

3.4 Decisão de pouso

A função `avaliarPouso` traduz as funções booleanas da Seção 2 em uma estrutura condicional if, elif e else, na mesma ordem lógica do circuito:

```
if autPouso:
    return "AUTORIZADO", ...
elif falhaCritica:      # (not A and comb < 30) or not D
    return "ALERTA", ...
else:
    return "ESPERA", ...
```

A rotina `processarFila` retira um módulo por vez da frente da fila, aplica a decisão e o envia para a lista correspondente, até a fila esvaziar. Trecho da execução da demonstração completa (opção 7) no modo base:

```
Analisando M05 - Suporte Vita
checks: comb=True atm=True area=True sens=False massa=True
Decisao: ALERTA OPERACIONAL
Motivo: Modulo exige revisao antes de nova tentativa de pouso.

Analisando M03 - Lab Ares
checks: comb=False atm=True area=True sens=True massa=True
Decisao: MODULO EM ESPERA
Motivo: Pouso adiado ate normalizacao das condicoes.

Analisando M04 - Carga Atlas
checks: comb=False atm=True area=True sens=True massa=True
Decisao: ALERTA OPERACIONAL
```

Resultado final do modo base: M02 e M01 pousados; M03 em espera (combustível abaixo do mínimo, mas não crítico); M05 (sensores) e M04 (combustível crítico) em alerta.

4. Modelagem matemática da descida

Para apoiar a decisão sobre o acionamento dos retropropulsores, modelamos a altura de um módulo durante a aproximação por uma função quadrática:

$$h(t) = h0 - a \cdot t^2 \qquad \text{com} \quad a = h0 / T^2$$

Os parâmetros, definidos no início do código, são h0 = 1.200 m (altura inicial da aproximação), T = 80 s (tempo até a altura chegar a zero) e, portanto, a = 1.200 / 80² = 0,1875 m/s². A variável t é o tempo em segundos e h(t), a altura em metros. A função `mostrarModeloDescida` imprime h(t) a cada 20 s e procura o primeiro instante em que a altura fica abaixo do limite de 550 m (`altRetroM`).

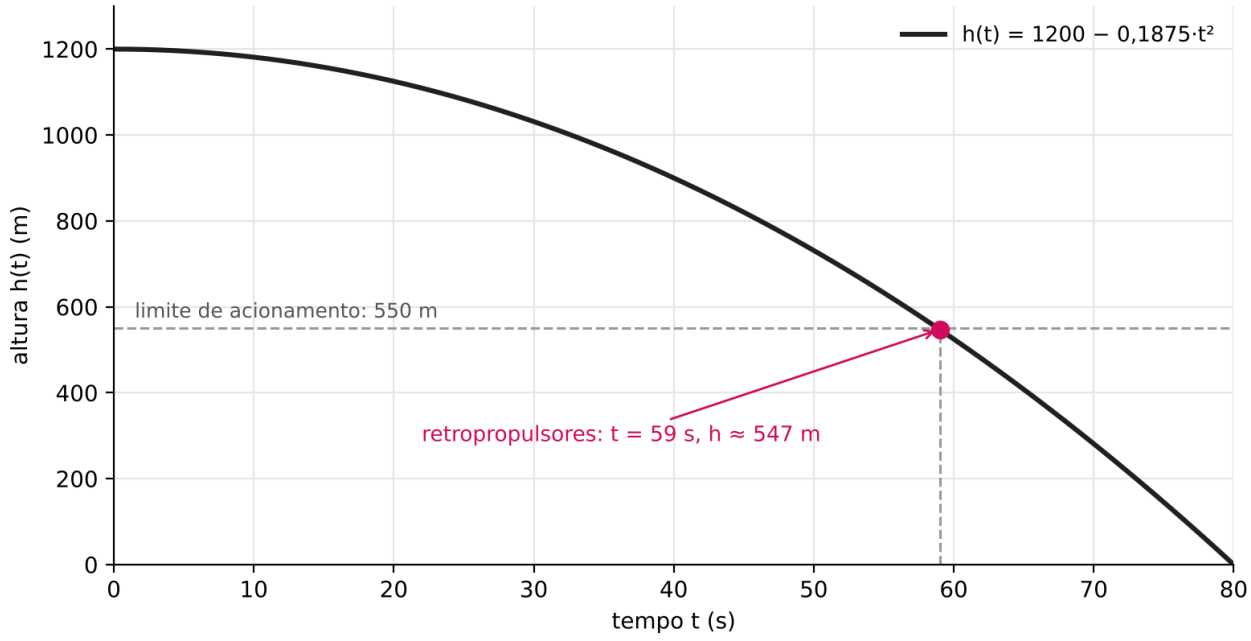


Figura 2 – Altura do módulo em função do tempo, com o limite de acionamento dos retropropulsores

t (s)	0	20	40	59	60	80
h(t) (m)	1.200,00	1.125,00	900,00	547,31	525,00	0,00

Análise qualitativa. O gráfico é um ramo de parábola com concavidade para baixo e vértice em t = 0, onde a altura é máxima. A curva começa quase horizontal e fica cada vez mais inclinada, porque a taxa de descida cresce com o tempo: a velocidade vertical é h'(t) = −2a·t, negativa porque a altura diminui; em módulo, |v(t)| = 2a·t vale 7,5 m/s em t = 20 s, cerca de 22 m/s em t = 59 s e 30 m/s em t = 80 s. Resolvendo h(t) = 550, obtemos t = T·√(1 − 550/h0) ≈ 58,9 s, e o programa sugere acionar os retropropulsores em t = 59 s, a 547,31 m.

Efeito dos parâmetros. Aumentar T (descida mais longa) reduz a e torna a curva mais suave, adiando o ponto de acionamento; reduzir T faz o contrário. Aumentar h0 mantendo T aumenta a: o módulo cruza o limite de 550 m mais tarde, porém mais rápido; diminuir h0 antecipa o cruzamento. Elevar o limite de 550 m antecipa o acionamento e aumenta a margem de segurança, ao custo de mais combustível gasto na frenagem.

Limites do modelo. A função representa a fase de queda antes da frenagem: nela a velocidade só aumenta, e o módulo chegaria ao solo a 30 m/s se nada fosse feito. Por isso, o valor de interesse não é o instante de contato, mas o instante de decisão: o modelo indica quando os retropropulsores devem entrar para reduzir essa velocidade. O comportamento após o acionamento não é modelado e seria um passo natural em uma próxima versão.

5. Contexto histórico e arquitetura

O MGPEB depende de uma ideia que atravessa toda a história da computação: chaves que ligam e desligam, organizadas logicamente para resolver problemas lógicos e aritméticos. As portas AND, OR, NOT e NOR da Figura 1 são exatamente esse tipo de arranjo. O que mudou ao longo das gerações foi a tecnologia da chave: engrenagens nas calculadoras mecânicas, relés nos primeiros computadores eletromecânicos, válvulas na primeira geração, transistores na segunda e circuitos integrados a partir da terceira, com graus crescentes de integração (LSI, VLSI e ULSI).

A máquina analítica de Charles Babbage, descrita em 1837, já reunia três fundamentos que reconhecemos no protótipo: entrada de dados (hoje, os modos base, randômico e manual), uma memória para guardar e reutilizar valores (as listas e a pilha) e a execução sequencial de operações (o menu e o processamento da fila). Os primeiros circuitos de relés, como os usados nos computadores de Konrad Zuse, já mostravam que uma soma binária pode ser feita com as mesmas portas que usamos: o bit de soma é um XOR e o "vai um" é um AND. Os computadores de propósito geral da primeira geração, como o ENIAC, com cerca de 30 toneladas e 17.840 válvulas, ocupavam salas inteiras; a miniaturização prevista pela Lei de Moore, que observou o número de transistores dobrar periodicamente, é o que permite hoje embarcar um sistema de decisão dentro de um módulo de pouso.

Um sistema embarcado em Marte, porém, trabalha com limitações que um computador pessoal não tem: memória e capacidade de processamento restritas, energia disputada com o suporte à vida, exposição à radiação e a impossibilidade de manutenção imediata. Essas limitações orientaram as escolhas do protótipo:

- estruturas lineares simples (listas, fila e pilha), sem bibliotecas externas;
- algoritmos simples e de custo previsível (busca linear e insertion sort), adequados ao tamanho pequeno da frota;
- regras booleanas explícitas e determinísticas, fáceis de verificar e de testar, o que é essencial quando um erro não pode ser corrigido a distância;
- registro dos alertas em pilha, para que a revisão comece pelo problema mais recente.

Em contrapartida, o protótipo faz cópias dos módulos para preservar o cadastro original, o que consome memória extra; em um sistema embarcado real, essa decisão seria reavaliada.

6. Governança ambiental, social e corporativa na base Aurora Siger

Introdução

O pouso dos módulos marca a passagem de "chegar a Marte" para "começar a viver em Marte". A partir desse momento, as decisões da colônia deixam de ser apenas técnicas e passam a afetar um ambiente ainda intocado, as pessoas que nele vão viver e trabalhar e a confiança em quem decide. Por isso, adotamos como referência os princípios ESG (*Environmental, Social and Governance*) e a definição de desenvolvimento sustentável do Relatório Brundtland: atender às necessidades do presente sem comprometer a capacidade das gerações futuras de atenderem às suas. Em Marte, essas "gerações futuras" são os próximos colonos e a própria ciência, que depende de um planeta preservado para ser estudado.

Critérios para a escolha da área de pouso

A escolha da área de pouso deve seguir uma abordagem preventiva, como recomenda o princípio 7 do Pacto Global: diante da incerteza sobre os impactos, evita-se o dano antes de remediá-lo. Na prática, propomos quatro critérios, além da segurança do pouso: (1) evitar regiões de alto interesse científico, preservando-as para pesquisa; (2) preferir áreas próximas a recursos úteis, como gelo e boa incidência solar, para reduzir deslocamentos e consumo de energia; (3) concentrar os pousos em uma área delimitada, limitando a extensão do solo alterado; e (4) registrar o estado original da área antes da ocupação, para permitir comparações futuras. O MGPEB já incorpora parte dessa lógica: o pouso só é autorizado quando a área está livre (`areaCheck`) e as condições atmosféricas são seguras (`atmCheck`), o que mostra que regras ambientais podem ser traduzidas em condições objetivas de decisão.

Gestão sustentável de energia, resíduos e recursos locais

Na dimensão ambiental, os recursos de Marte são escassos e, em grande parte, não renováveis na escala de tempo da colônia. A gestão proposta se apoia em três Objetivos de Desenvolvimento Sustentável da Agenda 2030. Para a energia (ODS 7), a base deve priorizar a geração solar e a eficiência, ligando primeiro os sistemas de suporte à vida; não por acaso, o módulo de energia está entre os primeiros da fila de pouso. Para a água (ODS 6), o gelo extraído deve ser tratado como recurso compartilhado, com medição do consumo e reaproveitamento em circuito fechado. Para os resíduos e a produção (ODS 12), vale a lógica da economia circular: reparar, reutilizar e reciclar materiais antes de descartá-los, já que não há reposição rápida vinda da Terra. Esses compromissos precisam ser acompanhados por indicadores, como energia gerada e consumida, água reaproveitada e resíduos reciclados, pois declarar uma base "sustentável" sem medir resultados seria uma forma de *greenwashing*.

Governança, transparência e participação

Na dimensão social e de governança, uma colônia isolada concentra poder em poucas pessoas e em sistemas automatizados, o que aumenta a importância de regras claras. Propomos: (1) decisões automatizadas com regras públicas e auditáveis, como as expressões booleanas do MGPEB, que qualquer colono pode ler e verificar; (2) registro de todas as ocorrências, como já faz a pilha de alertas do sistema, que guarda o módulo, o motivo, o combustível e o horário de chegada à órbita de cada alerta; (3) relatórios periódicos de desempenho ambiental e social, inspirados no modelo da Global Reporting Initiative (GRI); (4) um conselho com representantes das diferentes funções da base, para que decisões sobre uso de recursos sejam discutidas e não apenas impostas; e (5) condições de trabalho seguras, jornadas justas e combate à corrupção, em linha com os princípios de trabalho e o princípio 10 do Pacto Global. A responsabilidade final continua humana: o sistema recomenda, mas as pessoas respondem pelas decisões.

Conclusão

A reflexão mostra que o MGPEB não é apenas uma ferramenta de pouso, mas o primeiro passo de uma governança para a colônia. Ao combinar as três dimensões do ESG, o projeto busca o equilíbrio do *triple bottom line* (pessoas, planeta e resultado): uma base tecnicamente viável, que protege as pessoas, usa os recursos com responsabilidade e torna suas decisões transparentes. Em uma fronteira como Marte, que exige governança local compatível com os acordos internacionais, esses princípios são um compromisso voluntário, como o Pacto Global entre as empresas, e valem desde o primeiro pouso.

7. Conclusão

O MGPEB integra os conteúdos da Fase 2 em um único problema. As portas lógicas definem as regras de decisão; o Python as executa com estruturas condicionais e laços; as estruturas lineares organizam o fluxo dos módulos; a busca e a ordenação priorizam o pouso; a função quadrática indica o momento de acionar os retropropulsores; a história da computação explica as limitações do hardware embarcado; e os princípios ESG orientam o uso responsável da tecnologia e dos recursos de Marte.

No modo base, dois módulos pousaram, um ficou em espera e dois foram retidos em alerta por motivos distintos (falha de sensores e combustível crítico). O caso do M03, que aguarda com 33% de combustível enquanto o M04 vai para revisão com 28%, demonstra que as regras distinguem um problema grave de uma simples espera. Como evoluções, identificamos: modelar a fase de frenagem após o acionamento dos retropropulsores e substituir a lista da fila por uma estrutura com retirada em

tempo constante caso a frota cresça.

Anexo A – Estruturas de dados utilizadas

Estrutura	Variável	Tipo	Operações usadas	Papel na simulação
Fila principal	filaPouso	Fila (FIFO) sobre list	append para inserir; pop(0) para retirar da frente	módulos aguardando avaliação, na ordem de prioridade
Lista de pousados	modsPousados	Lista	append	módulos com pouso autorizado
Lista de espera	modsEspera	Lista	append	módulos com pouso adiado
Lista de alerta	modsAlerta	Lista	append	módulos que exigem revisão
Pilha de alertas	pilhaAlertas	Pilha (LIFO) sobre list	append para empilhar; pop para desempilhar	registro de alertas; o mais recente é revisado primeiro
Cadastro	todosMods	Lista de dicionários	percurso com for	base para buscas e para a carga da fila

Fila (FIFO). A função `carregarFila` insere os cinco módulos com append, e `prepararFilaOrdenada` substitui o conteúdo pela versão ordenada: M02, M01, M05, M03, M04. Em `processarFila`, o laço `while len(filaPouso) > 0` executa `filaPouso.pop(0)`, que retira sempre o módulo da frente, como o dequeue de uma fila. Assim, a primeira a sair é M02 e a última é M04. Em uma lista Python, `pop(0)` tem custo O(n), pois os demais elementos são deslocados; para cinco módulos o custo é desprezível, e em uma frota grande `collections.deque` faria a mesma operação em O(1).

Listas auxiliares. Depois da decisão, cada módulo vai para exatamente uma lista. No modo base: modsPousados = [M02, M01]; modsEspera = [M03]; modsAlerta = [M05, M04]. Essas listas permitem montar o resumo final (opção 5) sem reprocessar a fila.

Pilha (LIFO). A função `registrarAlerta` empilha, com append, um registro com id, nome, motivo, combustível e horário de órbita. No modo base, M05 é empilhado primeiro e M04 depois, ficando no topo. Para exibir a pilha sem destruí-la, `mostrarPilhaAlertas` copia os itens para uma pilha temporária e desempilha dessa cópia com pop; por isso, a exibição mostra primeiro M04 e depois M05, e a pilha original permanece intacta para novas consultas.

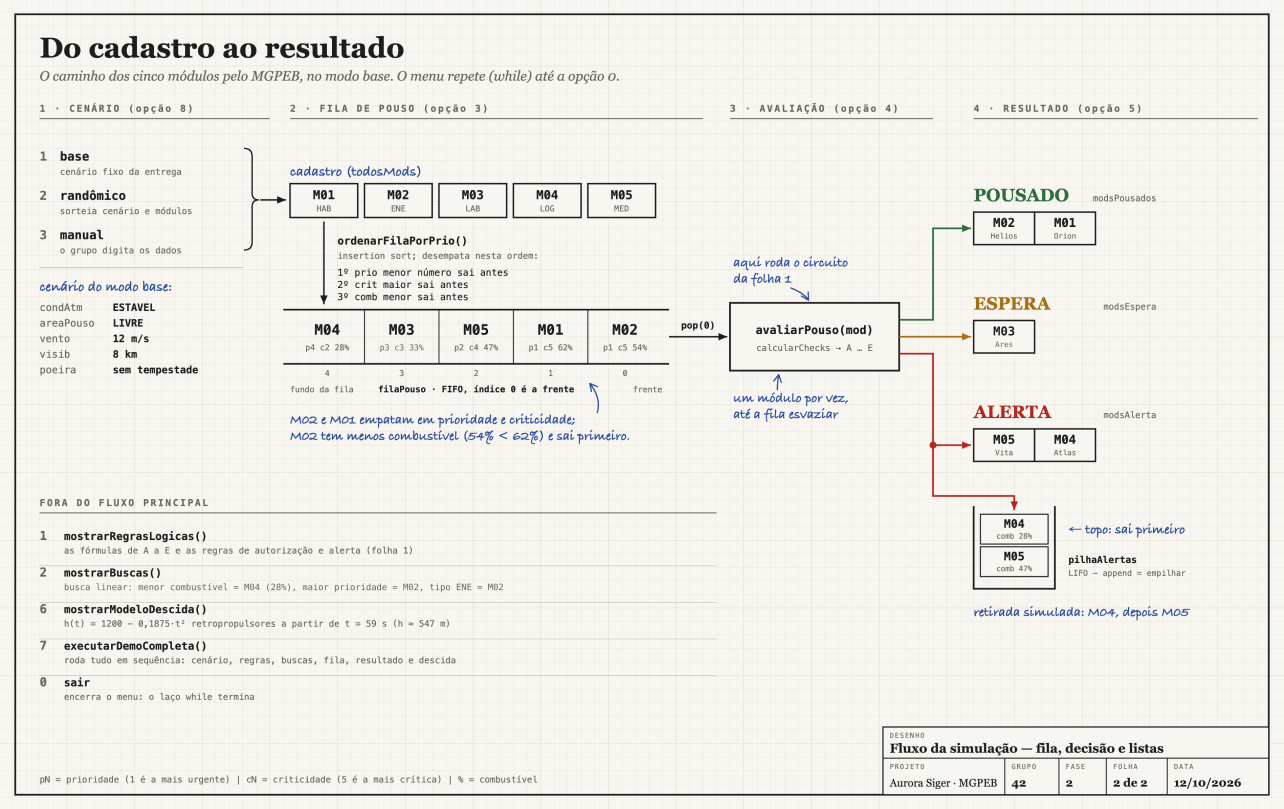


Figura 3 – Fluxo da simulação: cenário, fila ordenada, avaliação, listas auxiliares e pilha de alertas